

```
<vehicle depart="21" id="v15" route="rou01"
type="BoogyB" />
<vehicle depart="22" id="v16" route="rou01"
type="BoogyB" />
<vehicle depart="23" id="v17" route="rou02"
type="BoogyB" />
<vehicle depart="24" id="v18" route="rou02"
type="BoogyB" />
<vehicle depart="27" id="v19" route="rou02"
type="BoogyB" />
</routes>
```

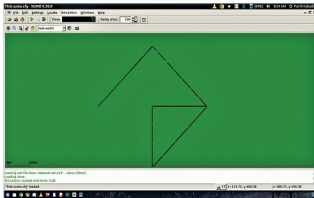


Figure 2: Simulation in the SUMO GUI

Files *first.rou.xml* and *network.net.xml* are required to generate configuration files for simulating the traffic on the network. The following configuration file is generated, which is then used to simulate the scenario:

```
first.sumo.cfg
```

```
<configuration>
  <input>
    <net-file value="network.net.xml"/>
    <route-files value="first.rou.xml"/>
  </input>
  <time>
    <begin value="0"/>
    <end value="150"/>
  </time>
  <time-to-teleport value="-1"/>
</configuration>
```

To simulate the scenario, the following command is required to be fired from the terminal:

```
$sumo -c first.sumo.cfg
```

To visualise the simulation in the SUMO GUI, use the following command:

```
$sumo-gui -c first.sumo.cfg
```

Generating traces

SUMO trace files can be generated with the help of a .cfg file (configuration file), as follows:

```
$sumo -c first.sumo.cfg -fcd-output trace.xml
```

Traces of the vehicles' position are dumped into the XML file. These SUMO traces can be used to provide mobility to nodes in the ns-3 simulator. The ns-3 simulator provides the ns-2 mobility helper class, which can be used to read the movements of nodes from trace files. To make use of the SUMO traces generated from the SUMO simulator, the XML file needs to be converted to a TCL file, which then is provided to the ns-2 mobility helper. SUMO provides tools like *traceExporter.py* and *traceExporter.jar* to convert SUMO trace to different file formats.

```
$python $SUMO_HOME/tools/traceExporter.py -fcd-input trace.
xml -ns2mobility-output mobility.tcl
```

mobility.tcl generated above contains the position and the velocities of the nodes (vehicles).

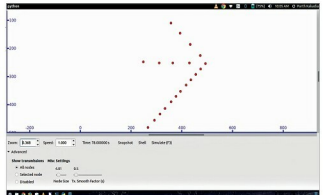


Figure 3: Simulation in pyViz

mobility.tcl as the argument along with a number of nodes in the scenario and the duration of the simulation. At the end of the simulation, it generates a log file. The command for executing the same is given below:

```
$.waf --run "ns2-mobility-trace --traceFile=path_to_
tracefile/mobility.tcl --nodeName=18 --duration=200.0
--logFile=ns2-mob.log" --vis
```

--vis renders the simulation in Python Visualizer.

Visualising the scenario in NetAnim: A NetAnim compatible XML file can be generated by adding the following lines of code in the file *ns2-mobility-trace.cc* (*ns-dev/src/mobility/examples/*).

Include the header file:

```
#include "ns3/netanim-module.h"
```